Assignment Cover– be sure to keep a copy of all work submitted

To be completed by student – PLEASE PRINT CLEARLY

Name: AMZAR FAIZZUDDIN BIN MOHD HAMDAN		
ID Number: AM2508020090		
Lecturer: NURAIN NABILA BINTI BAHAROM		Lab group / Tutorial group / Tutor (if applicable)
Course and Course Code: SWC3633		Submission Date: 20/10/2026
Assignment No. / Title PROJECT		Extension & Late submission: Allowed / Disallowed
Assignment type: ASSIGNMENT	% of Assignment Mark	Returning Date:

Penalties:

1. 10% of the original mark will be deducted for every one-week period after the submission date
2. No work will be accepted after two weeks of the deadline
3. If you were unable to submit the coursework on time due to extenuating circumstances you may be eligible for an extension
4. Extension will not exceed one week

Declaration: I/we the undersigned confirm that I/we have read and agree to abide by these regulations on plagiarism and cheating. I/we confirm that this piece of work is my/our own. I/we consent to appropriate storage of our work for checking to ensure that there is no plagiarism/ academic cheating.

Signature: 

Full Name: AMZAR FAIZZUDDIN BIN MOHD HAMDAN

This section may be used for feedback or other information

1. System Overview

EventEase is a web application prototype for discovering events and managing ticket bookings. Its intended roles are Administrator, Organiser and Customer. Customers can browse published events, register/login, create and cancel bookings, and inspect booking status. Organisers manage events they own. Administrators have wider system-management privileges. The frontend uses HTML, CSS and JavaScript Fetch; the backend is a PHP JSON REST API backed by MySQL.

2. Database Design

The relational schema includes users, venues, events, bookings, payments and api_tokens. Primary keys identify rows; foreign keys connect organisers to events, venues to events, customers to bookings, events to bookings, bookings to payments and users to tokens.

Entity Relationship Diagram (ERD)

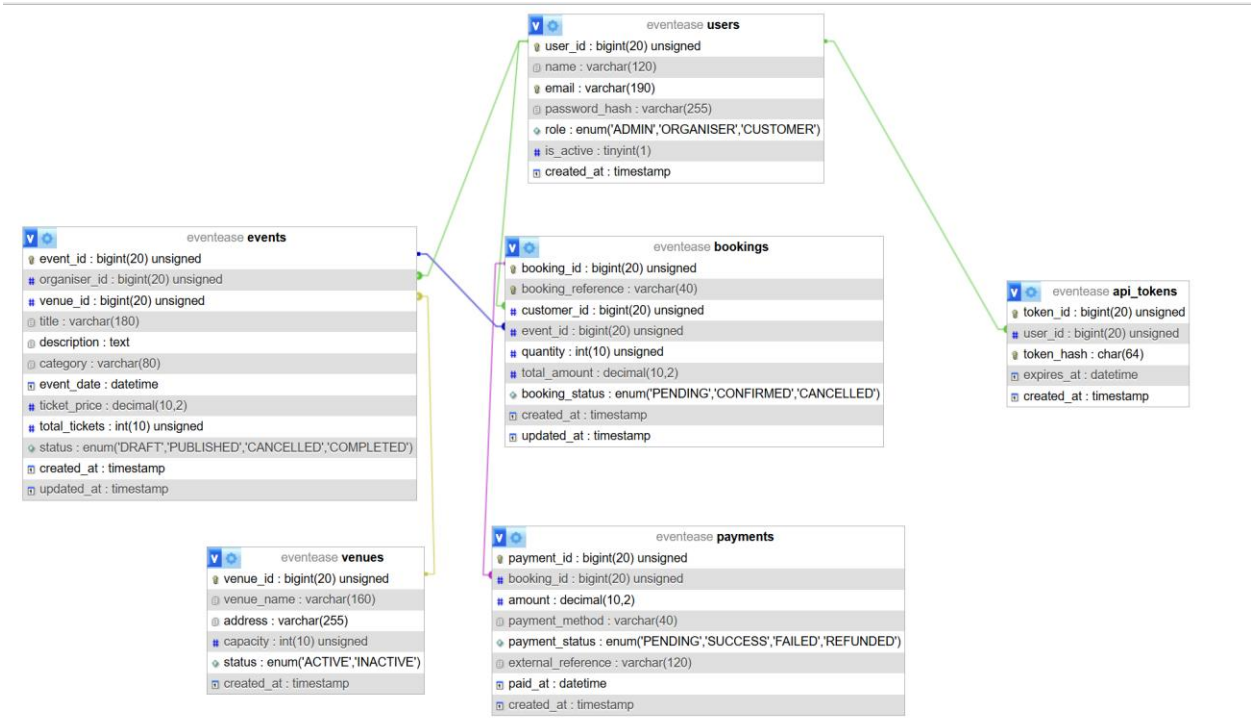


Figure 1: Entity Relationship Diagram (ERD) of the EventEase Database

Data Definition Language (DDL) - database/schema.sql

The DDL script creates the eventease database and its six tables. Each table uses an auto-increment BIGINT primary key. Foreign keys link events to their organiser and venue, bookings to the customer and event, payments to the booking, and API tokens to the user. Data integrity is enforced with NOT NULL, UNIQUE (email, booking_reference, token_hash), ENUM status values and CHECK constraints for positive capacity, ticket quantity and non-negative prices and amounts. Indexes support event searches by date, category and status, and booking lookups by customer and by event status.

```
CREATE DATABASE IF NOT EXISTS eventease
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE eventease;

CREATE TABLE IF NOT EXISTS users (
  user_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(120) NOT NULL,
  email VARCHAR(190) NOT NULL UNIQUE,
```

```
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('ADMIN','ORGANISER','CUSTOMER') NOT NULL DEFAULT 'CUSTOMER',
  is_active TINYINT(1) NOT NULL DEFAULT 1,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS venues (
  venue_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  venue_name VARCHAR(160) NOT NULL,
  address VARCHAR(255) NOT NULL,
  capacity INT UNSIGNED NOT NULL,
  status ENUM('ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT chk_venue_capacity CHECK (capacity > 0)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS events (
  event_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  organiser_id BIGINT UNSIGNED NOT NULL,
  venue_id BIGINT UNSIGNED NOT NULL,
  title VARCHAR(180) NOT NULL,
  description TEXT NULL,
  category VARCHAR(80) NOT NULL,
  event_date DATETIME NOT NULL,
  ticket_price DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  total_tickets INT UNSIGNED NOT NULL,
  status ENUM('DRAFT','PUBLISHED','CANCELLED','COMPLETED') NOT NULL DEFAULT 'DRAFT',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_events_organiser FOREIGN KEY (organiser_id) REFERENCES users(user_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_events_venue FOREIGN KEY (venue_id) REFERENCES venues(venue_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_event_price CHECK (ticket_price >= 0),
  CONSTRAINT chk_event_tickets CHECK (total_tickets > 0),
  INDEX idx_events_date (event_date),
  INDEX idx_events_category (category),
  INDEX idx_events_status (status)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS bookings (
  booking_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  booking_reference VARCHAR(40) NOT NULL UNIQUE,
  customer_id BIGINT UNSIGNED NOT NULL,
  event_id BIGINT UNSIGNED NOT NULL,
  quantity INT UNSIGNED NOT NULL,
  total_amount DECIMAL(10,2) NOT NULL,
  booking_status ENUM('PENDING','CONFIRMED','CANCELLED') NOT NULL DEFAULT 'PENDING',
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_bookings_customer FOREIGN KEY (customer_id) REFERENCES users(user_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_bookings_event FOREIGN KEY (event_id) REFERENCES events(event_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_booking_quantity CHECK (quantity > 0),
  CONSTRAINT chk_booking_amount CHECK (total_amount >= 0),
  INDEX idx_bookings_customer (customer_id),
  INDEX idx_bookings_event_status (event_id, booking_status)
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS payments (
  payment_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  booking_id BIGINT UNSIGNED NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  payment_method VARCHAR(40) NOT NULL DEFAULT 'DEMO',
  payment_status ENUM('PENDING','SUCCESS','FAILED','REFUNDED') NOT NULL DEFAULT 'PENDING',
  external_reference VARCHAR(120) NULL,
  paid_at DATETIME NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_payments_booking FOREIGN KEY (booking_id) REFERENCES bookings(booking_id)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_payment_amount CHECK (amount >= 0),
  INDEX idx_payments_booking (booking_id)
) ENGINE=InnoDB;

-- Security: store hashed tokens rather than raw bearer tokens.
CREATE TABLE IF NOT EXISTS api_tokens (
  token_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
```

```
  user_id BIGINT UNSIGNED NOT NULL,
  token_hash CHAR(64) NOT NULL UNIQUE,
  expires_at DATETIME NOT NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_tokens_user FOREIGN KEY (user_id) REFERENCES users(user_id)
    ON UPDATE CASCADE ON DELETE CASCADE,
  INDEX idx_tokens_expiry (expires_at)
) ENGINE=InnoDB;
```

Data Manipulation Language (DML) - database/seed.sql

The DML script inserts five sample records into each of the users, venues, events, bookings and payments tables. All seeded accounts share the classroom-only demo password documented in the README, stored as a bcrypt hash generated with PHP password_hash(). The api_tokens table is filled automatically when users log in.

```
USE eventease;

-- Demo password for all seeded users: EventEaseDemo!2026
-- Hash generated using PHP password_hash() with PASSWORD_DEFAULT.
-- Replace these demo accounts/passwords before any public deployment.
INSERT INTO users (user_id, name, email, password_hash, role) VALUES
(1, 'System Administrator', 'admin@eventease.test', '$2y$12$SFEw/qk8Tg9BDvgnrTcDZO2zvyPbsys81Ynv.xHy.6O8ID2W9YURa', 'ADMIN'),
(2, 'Aina Organiser', 'aina@eventease.test', '$2y$12$SFEw/qk8Tg9BDvgnrTcDZO2zvyPbsys81Ynv.xHy.6O8ID2W9YURa', 'ORGANISER'),
(3, 'Ravi Organiser', 'ravi@eventease.test', '$2y$12$SFEw/qk8Tg9BDvgnrTcDZO2zvyPbsys81Ynv.xHy.6O8ID2W9YURa', 'ORGANISER'),
(4, 'Sofia Customer', 'sofia@eventease.test', '$2y$12$SFEw/qk8Tg9BDvgnrTcDZO2zvyPbsys81Ynv.xHy.6O8ID2W9YURa', 'CUSTOMER'),
(5, 'Daniel Customer', 'daniel@eventease.test', '$2y$12$SFEw/qk8Tg9BDvgnrTcDZO2zvyPbsys81Ynv.xHy.6O8ID2W9YURa', 'CUSTOMER');

INSERT INTO venues (venue_id, venue_name, address, capacity, status) VALUES
(1, 'KL Convention Studio', 'Kuala Lumpur', 500, 'ACTIVE'),
(2, 'Petaling Jaya Hall', 'Petaling Jaya, Selangor', 250, 'ACTIVE'),
(3, 'Shah Alam Creative Space', 'Shah Alam, Selangor', 150, 'ACTIVE'),
(4, 'Cyberjaya Auditorium', 'Cyberjaya, Selangor', 400, 'ACTIVE'),
(5, 'Kajang Community Centre', 'Kajang, Selangor', 100, 'ACTIVE');

INSERT INTO events
(event_id, organiser_id, venue_id, title, description, category, event_date, ticket_price, total_tickets, status) VALUES
(1, 2, 1, 'Digital Innovation Summit', 'A sample technology and innovation event.', 'Technology', '2027-03-10 09:00:00',
80.00, 300, 'PUBLISHED'),
(2, 2, 2, 'Creative Business Workshop', 'A practical workshop for small businesses.', 'Business', '2027-03-18 10:00:00',
35.00, 150, 'PUBLISHED'),
(3, 3, 3, 'Community Arts Festival', 'A community arts and culture programme.', 'Arts', '2027-04-02 11:00:00', 20.00, 120,
'PUBLISHED'),
(4, 3, 4, 'Future Skills Conference', 'A conference about future-ready skills.', 'Education', '2027-04-15 09:30:00', 60.00,
250, 'PUBLISHED'),
(5, 2, 5, 'Local Food and Craft Fair', 'A local food and craft community event.', 'Community', '2027-05-01 10:00:00', 10.00,
80, 'DRAFT');

INSERT INTO bookings
(booking_id, booking_reference, customer_id, event_id, quantity, total_amount, booking_status) VALUES
(1, 'EE-DEMO-1001', 4, 1, 2, 160.00, 'PENDING'),
(2, 'EE-DEMO-1002', 5, 2, 1, 35.00, 'CONFIRMED'),
(3, 'EE-DEMO-1003', 4, 3, 3, 60.00, 'CANCELLED'),
(4, 'EE-DEMO-1004', 5, 4, 2, 120.00, 'PENDING'),
(5, 'EE-DEMO-1005', 4, 2, 2, 70.00, 'CONFIRMED');

INSERT INTO payments
(payment_id, booking_id, amount, payment_method, payment_status, external_reference, paid_at) VALUES
(1, 1, 160.00, 'DEMO', 'PENDING', NULL, NULL),
(2, 2, 35.00, 'DEMO', 'SUCCESS', 'DEMO-PAY-2002', '2027-02-01 12:00:00'),
(3, 3, 60.00, 'DEMO', 'REFUNDED', 'DEMO-PAY-2003', '2027-02-02 12:00:00'),
(4, 4, 120.00, 'DEMO', 'PENDING', NULL, NULL),
(5, 5, 70.00, 'DEMO', 'SUCCESS', 'DEMO-PAY-2005', '2027-02-03 12:00:00');
```

3. RESTful API Documentation

All endpoints are served under the base URL http://localhost/EventEase/backend/api/v1 and exchange JSON. Protected endpoints require the header Authorization: Bearer <JWT> obtained from POST /auth/login. The administrator metrics endpoint additionally requires the X-API-Key header. Every response uses a consistent envelope: successful responses return {"success": true, "data": ...} and failed responses return {"success": false, "error": {"code": ..., "message": ...}}.

3.1 Endpoint summary

Method	URL	Access	Request parameters / body	Success	Main error codes
POST	/auth/register	Public	name, email, password (min. 10 chars)	201 user_id	400 INVALID_JSON, 422 INVALID_EMAIL / WEAK_PASSWORD, 409 EMAIL_EXISTS
POST	/auth/login	Public	email, password	200 access_token, expires_at, user	401 LOGIN_FAILED, 422, 429 RATE_LIMITED
GET	/auth/me	Any signed-in user	—	200 current user	401
POST	/auth/logout	Any signed-in user	—	200 token revoked	401
GET	/events	Public (manage=1: Admin, Organiser)	page, limit (max 50), q, category, sort (event_date | title | ticket_price), direction, manage	200 data + meta {page, limit}	401, 403
GET	/events/{id}	Public for PUBLISHED events	—	200 event incl. tickets_available	404
POST	/events	Admin, Organiser	venue_id, title, category, event_date, ticket_price, total_tickets, description, status	201 event_id	422 INVALID_EVENT_DATA / INVALID_EVENT_DATE / INVALID_VENUE / CAPACITY_EXCEEDED, 403
PUT / PATCH	/events/{id}	Admin, owning Organiser	all event fields incl. status	200	403, 404, 422
DELETE	/events/{id}	Admin	—	204	404, 409 RESOURCE_IN_USE
GET	/venues, /venues/{id}	Public	—	200	404
POST	/venues	Admin	venue_name, address, capacity, status	201 venue_id	422 INVALID_CAPACITY / INVALID_VENUE_STATUS
PUT / PATCH	/venues/{id}	Admin	venue_name, address, capacity, status	200	404, 422
DELETE	/venues/{id}	Admin	—	204	404, 409 RESOURCE_IN_USE
GET	/users, /users/{id}	Admin	—	200 (no password hashes)	401, 403, 404
POST	/users	Admin	name, email, password, role	201 user_id	422 INVALID_USER_DATA
PUT / PATCH	/users/{id}	Admin	name, email, role, is_active	200	404, 422
DELETE	/users/{id}	Admin	—	204	409 SELF_DELETE_BLOCKED / RESOURCE_IN_USE
GET	/bookings, /bookings/{id}	Customer (own), Organiser (own events), Admin (all)	—	200 bookings with payment status	401, 404
POST	/bookings	Customer	event_id, quantity (1–10)	201 booking_reference, PENDING	403, 404, 409 INSUFFICIENT_TICKETS, 422 INVALID_QUANTITY
PATCH	/bookings/{id}/cancel	Customer (own), Organiser, Admin	—	200 CANCELLED	404, 409 ALREADY_CANCELLED / CANCELLATION_CLOSED
PATCH	/bookings/{id}/demo-confirmation	Admin	—	200 CONFIRMED (simulated payment)	409 BOOKING_CANCELLED
GET	/bookings/{id}/qr	Booking owner, Organiser, Admin	—	200 image_base64 (QR PNG)	409 TICKET_NOT_CONFIRMED, 502

					QR_PROVIDER_ERROR
GET	/admin/metrics	Admin + X-API-Key	—	200 users, events, bookings counts	401 API_KEY_REQUIRED, 403

Design note: bookings are cancelled with PATCH /bookings/{id}/cancel instead of being deleted, so that booking and payment history is preserved. Cancelling a paid booking marks its payment as REFUNDED. The demo-confirmation endpoint simulates payment for coursework only; no real payment gateway is used.

3.2 Sample request and response

Login request (POST /auth/login):

```
{
  "email": "sofia@eventease.test",
  "password": "********"
}
```

Response format (200 OK):

```
{
  "success": true,
  "message": "Login successful.",
  "data": {
    "token_type": "Bearer",
    "access_token": "<JWT>",
    "expires_at": "<date-time>",
    "user": { "user_id": 4, "name": "Sofia Customer",
              "email": "sofia@eventease.test", "role": "CUSTOMER" }
  }
}
```

Booking request (POST /bookings, customer JWT):

```
{ "event_id": 1, "quantity": 1 }
```

Response format (201 Created):

```
{
  "success": true,
  "message": "Booking created and awaiting payment confirmation.",
  "data": { "booking_id": 12, "booking_reference": "EE-XXXXXXXXXX",
            "quantity": 1, "total_amount": 80, "booking_status": "PENDING" }
}
```

Validation error format (422 Unprocessable Entity):

```
{
  "success": false,
  "error": { "code": "INVALID_QUANTITY",
             "message": "Quantity must be an integer from 1 to 10." }
}
```

4. API Testing

The API was tested in Postman against the local XAMPP deployment using the exported collection EventEase API Tests (docs/EventEase.postman_collection.json). Collection variables hold the base URL and API key, and the login requests store the returned JWT in collection variables for later requests. The table below lists the tests whose results were captured; the screenshots are reproduced in Appendix A and Appendix B.

No.	Test case	Request	Expected	Recorded	Evidence

				result	
1	Public event list with pagination and sorting	GET /events?page=1&limit=10&sort=event_date&direction=ASC	200 OK, JSON list	200 OK	Appendix A
2	Public search	GET /events?q=Workshop&sort=title&direction=ASC	200 OK, matching events	200 OK	Appendix A
3	Protected endpoint without JWT	GET /bookings (no Authorization header)	401 AUTH_REQUIRED	401 Unauthorized	Appendix A
4	Customer views own bookings	GET /bookings (customer JWT)	200 OK	200 OK	Appendix A
5	Invalid booking quantity	POST /bookings {"event_id":1,"quantity":0}	422 INVALID_QUANTITY	422 Unprocessable Entity	Appendix A, Section 9
6	Registration with invalid email	POST /auth/register (email "not-an-email")	422 INVALID_EMAIL	422 Unprocessable Entity	Appendix A, Section 9
7	Admin metrics with JWT and API key	GET /admin/metrics (admin JWT + X-API-Key)	200 OK with counts	200 OK	Appendix A, Appendix B
8	Invalid or expired JWT	Protected request with an invalid token	401	401 Unauthorized	Appendix B
9	Administrator lists all bookings	GET /bookings (admin JWT)	200 OK	200 OK, Postman assertion passed (1/1)	Appendix B

In addition, the demonstration video shows the frontend flow of browsing events, booking tickets, viewing booking status and displaying a QR ticket for a confirmed booking. Venue and user management endpoints are documented in Section 3 but are not yet covered by separate Postman requests.

5. System Implementation

The backend uses PDO prepared statements for database access and a JSON router. Authentication is implemented with HS256 JWTs, expiry claims and a server-side hashed-token record for revocation. Cross-cutting middleware helpers provide request logging and login rate limiting. Public event list requests support pagination, search, category filtering and allow-listed sorting. The frontend consumes the API with Fetch and displays user feedback.

6. System Demonstration

The following screenshots document the running EventEase frontend. They show the login page, home page, published event listing, customer bookings and organiser event management.

Demonstration video: https://youtu.be/R7eWpEWpOoc. The video opens with the student name and ID, then shows the project structure, customer login with JWT, browsing events and booking tickets, booking status and the QR ticket from the QRServer API, administrator event management, and Postman requests showing a 401 response without a JWT and the paginated, sorted event list.

Figure 2: Login page

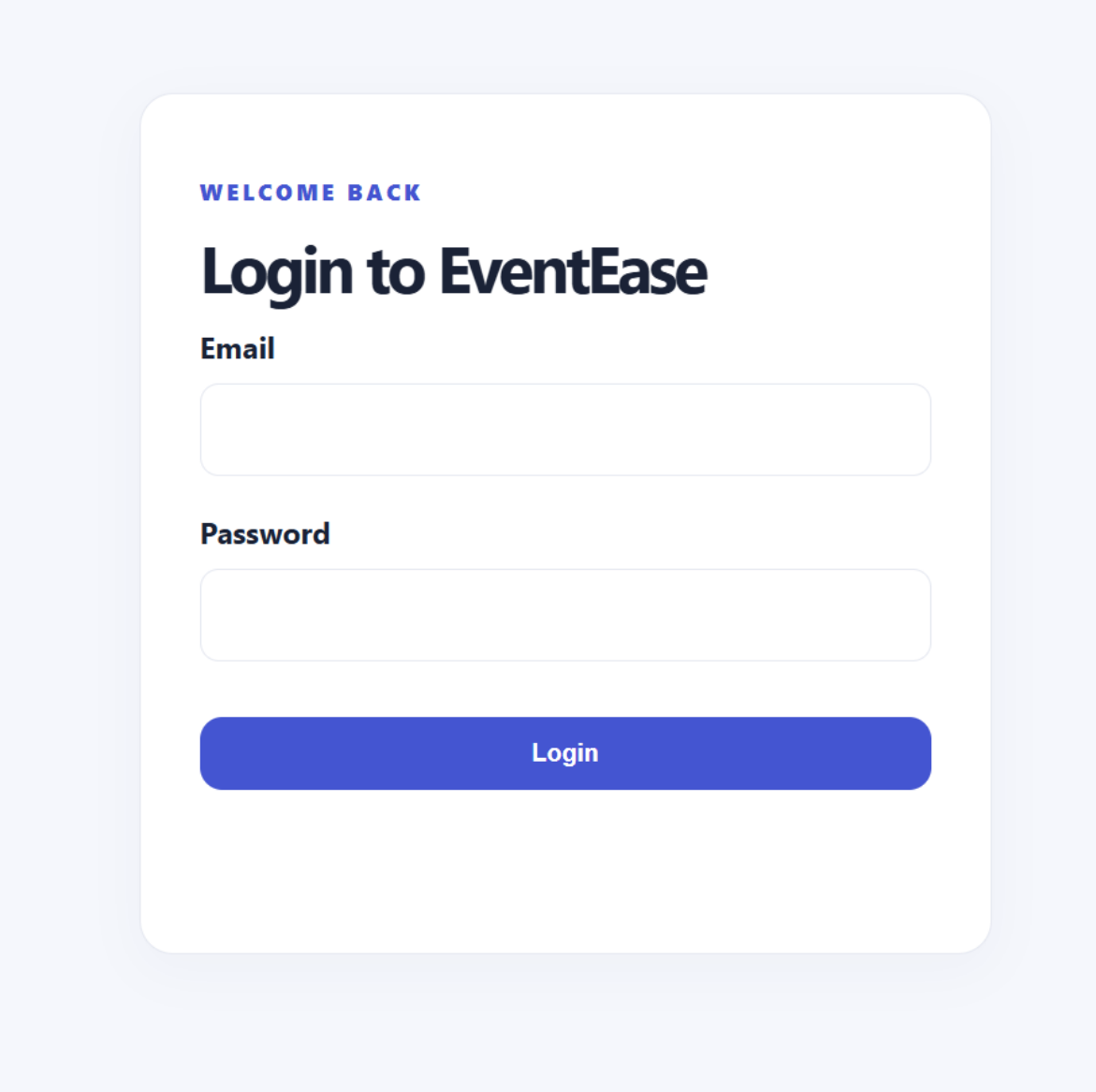


Figure 3: EventEase home page

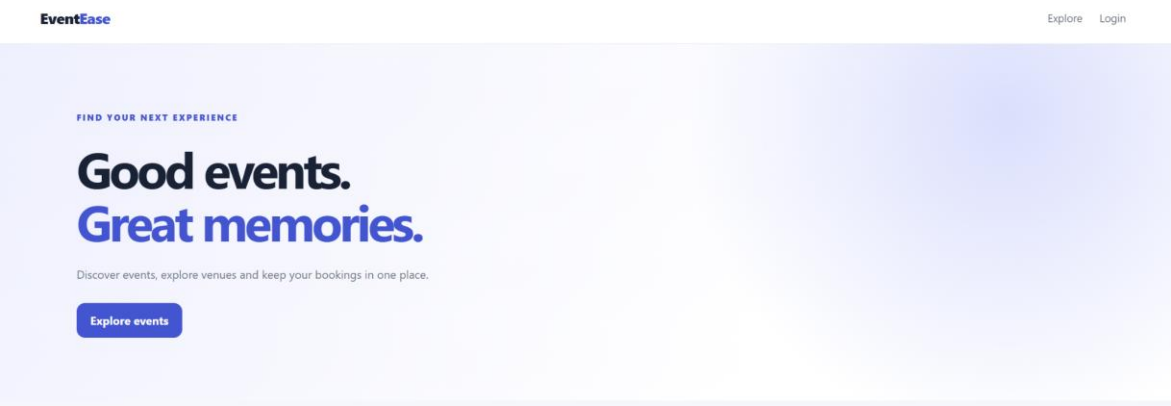


Figure 4: Explore Events page showing published events and ticket availability

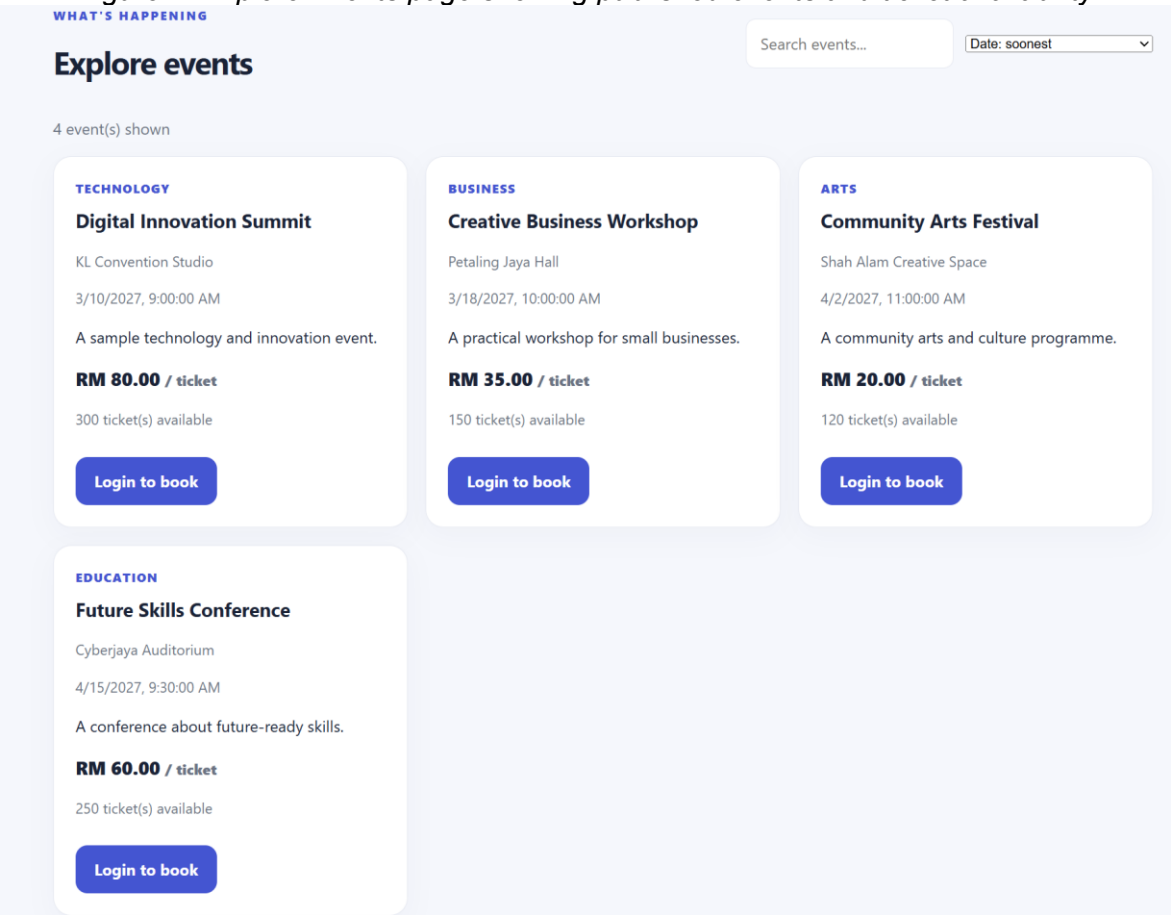


Figure 5: My Bookings page showing booking references, amounts and statuses

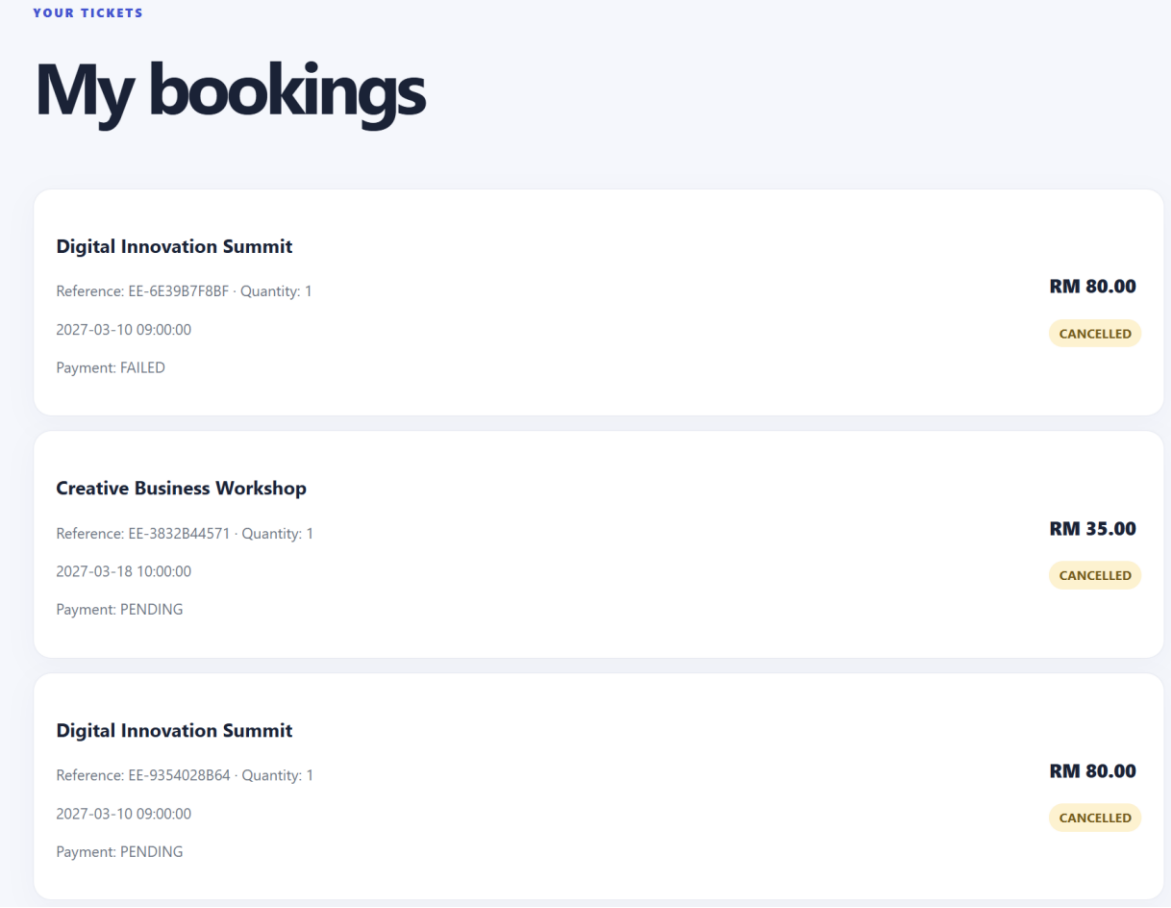


Figure 6: Navigation menu after login, including booking and event management options

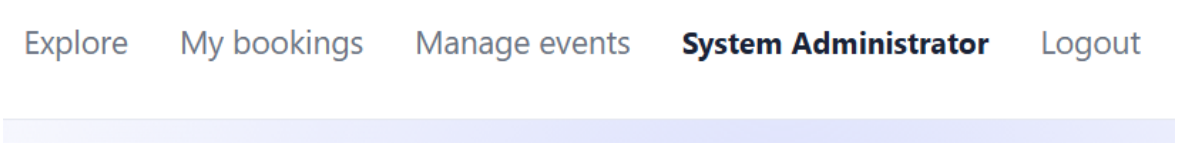


Figure 7: Organiser Manage Events page and Create Event form

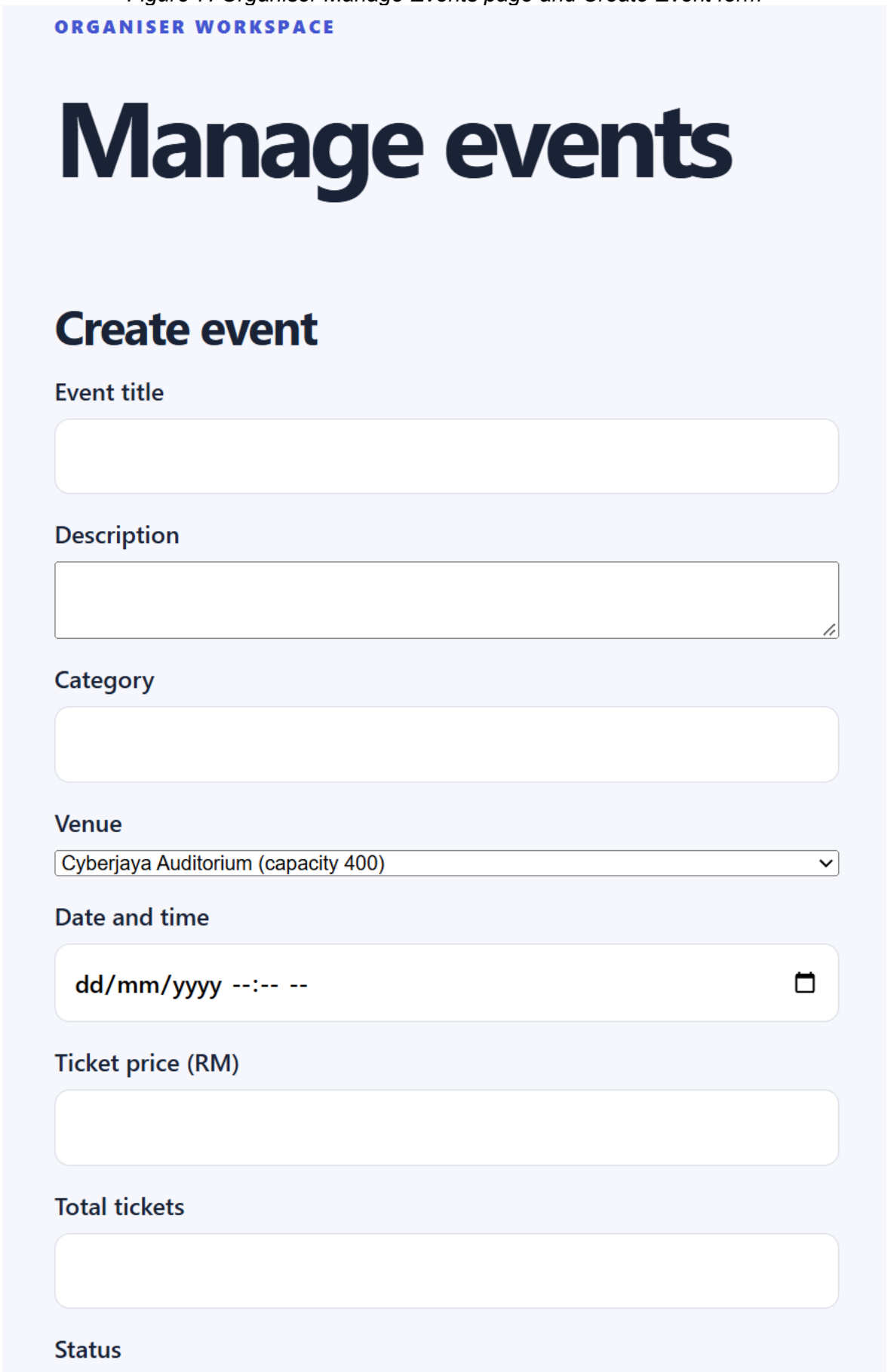
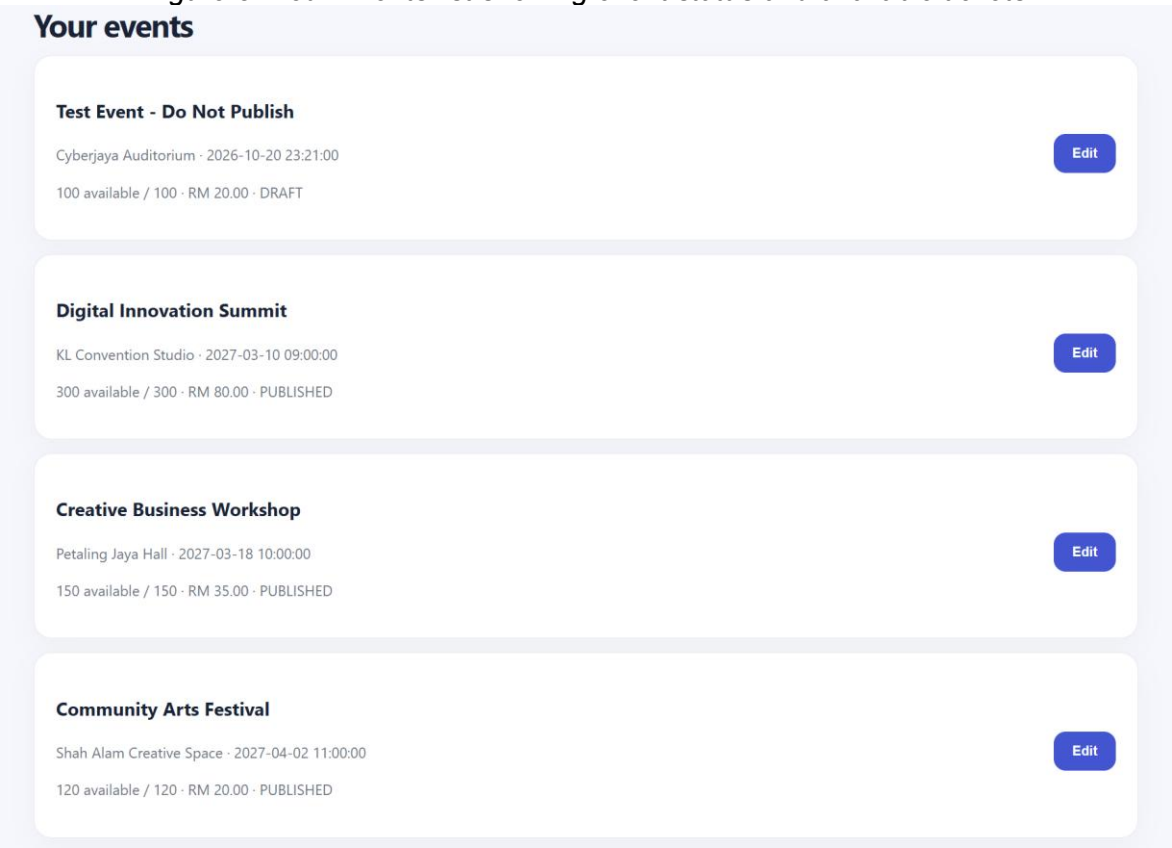


Figure 8: Your Events list showing event status and available tickets



7. Project Repository

GitHub repository: https://github.com/stomeyamzar/EventEase-Project-SWC3633. The repository contains the complete backend source code (backend/), the frontend (frontend/), the DDL script database/schema.sql and the DML script database/seed.sql with at least five sample records per core table, the ERD (docs/ERD.png and docs/ERD.md), the API documentation (docs/API_DOCUMENTATION.md), the exported Postman collection (docs/EventEase.postman_collection.json), this report in PDF format (docs/EventEase_Project_Report.pdf) and a README with setup and execution instructions. Secrets are excluded: backend/.env and log files are listed in .gitignore, only the template backend/.env.example is committed, and the API key in the published Postman collection is left blank.

8. Additional/Advanced Features

JWT authentication. Login issues an HS256-signed JWT containing the claims iss, sub, email, role, iat, exp and jti, valid for 8 hours. A SHA-256 hash of each token is stored in the api_tokens table. On every protected request the API verifies the signature and expiry, checks that the token hash still exists (so logout revokes the token), and confirms that the user is active and the role matches.

Role-based access control. Three roles are enforced on the server: Customers browse events and create, view and cancel their own bookings; Organisers create and edit only the events they own and see bookings for those events; Administrators manage users, venues and all events and bookings. Requests from the wrong role receive 403 FORBIDDEN, and records that belong to another user are reported as 404 so that their existence is not revealed.

API key protection. GET /admin/metrics requires both an administrator JWT and a valid X-API-Key header. The key is compared with hash_equals() and is loaded from the environment rather than written in the report.

Request validation and centralised error handling. Required fields, email format, password length, quantities, prices, dates, venue capacity and status values are validated before database writes, returning 422 with a specific error code. A single try/catch in the router converts database and unexpected errors into generic 409 or 500 JSON responses, so SQL messages are written to the server log instead of being returned to the client. Unknown routes return 404 and unsupported methods return 405.

Third-party API: goQR QRServer. GET /bookings/{id}/qr builds a JSON payload containing the booking reference, event title, event date and quantity, and sends it to https://api.qrserver.com/v1/create-qr-code/ using PHP cURL with a 4-second connection timeout and an 8-second total timeout. The QR code is only issued for CONFIRMED bookings (otherwise 409 TICKET_NOT_CONFIRMED). The PNG returned by QRServer is base64-encoded in the JSON response and displayed by the My Bookings page. If the provider fails, the API returns 502 QR_PROVIDER_ERROR.

Request logging. A shutdown handler writes one JSON line per request to backend/logs/api.log containing the time, HTTP method, path, status code, duration in milliseconds and client IP address. The log folder is excluded from the repository.

Login rate limiting. Each combination of client IP address and email is limited to 10 login attempts in a five-minute window; further attempts return 429 RATE_LIMITED.

Pagination, search, filtering and sorting. GET /events accepts page and limit (limit capped at 50), a keyword q that searches title, description and category, a category filter, and sort/direction parameters. Sort columns are taken from an allow-list so user input is never inserted into the ORDER BY clause.

Transactional booking. Booking creation runs inside a database transaction that locks the event row with SELECT … FOR UPDATE, recalculates booked tickets and rejects the request with 409 INSUFFICIENT_TICKETS if capacity would be exceeded, preventing overselling when two customers book at the same time.

9. Debugging and Performance Optimisation

The following issues were encountered while developing, testing and publishing the project. The fixes below describe the actions actually taken; no quantitative performance benchmark was recorded.

Issue 1 — Bearer token not accepted: Protected pages such as My Bookings kept redirecting to the login page because the API did not accept the Bearer token sent by the frontend. The root cause was a typing mistake. After correcting it, I logged in again, created a booking and confirmed that it appeared in My Bookings.

Issue 2 — Repository structure lost during the first GitHub upload: The first web upload placed every file in the repository root and dropped or renamed files whose names begin with a dot, so backend/.htaccess, which provides URL rewriting and forwards the Authorization header to PHP, was missing. A cloned copy would therefore not have run. The repository was recreated with the backend, frontend, database and docs folders, the missing files were added in separate commits, and a misspelt .htaccess filename was corrected.

Issue 3 — API key exposed in the exported Postman collection: Before publishing, the exported collection was found to contain the real X-API-Key value from the local .env file. The value was cleared in the published collection, .env and log files were added to .gitignore, and only the template backend/.env.example is committed.

Issue 4 — API input validation: Postman tests were used to verify that invalid inputs were rejected. A booking quantity outside the accepted range returned 422 Unprocessable Entity with INVALID_QUANTITY, and an invalid email address during registration returned 422 Unprocessable Entity with INVALID_EMAIL.

Issue 5 — Authentication and access control checks: A protected endpoint requested without a Bearer token returned 401 Unauthorized with AUTH_REQUIRED. Admin metrics returned 200 OK when tested with an administrator JWT and the required API key.

Performance optimisation note: No before-and-after load test or response-time benchmark was recorded, so this report does not claim a measured performance improvement. The implementation uses PDO prepared statements and pagination/search options as design measures; their performance impact was not separately benchmarked.

10. Individual Reflection and Contribution Record (200–300 words)

I completed EventEase individually with my lecturer's permission. I used an AI assistant to help generate parts of the code and documentation; my own work was running the system on XAMPP, testing the API in Postman and the browser, fixing problems, and preparing the repository, report and demonstration video.

My first challenge was authentication. Protected pages kept sending me back to the login page because the Bearer token was not being accepted. The cause turned out to be a simple typo, and after correcting it I confirmed the full flow by logging in, creating a booking and seeing it in My Bookings. A second challenge was publishing the project. My first GitHub upload flattened every file into one folder and lost the files whose names begin with a dot, including backend/.htaccess, which the API needs for routing and for passing the Authorization header. I recreated the repository with the correct structure and fixed the file names in separate commits. Before publishing I also noticed that my exported Postman collection contained my real API key, so I removed it and kept secrets only in the ignored .env file.

The most important lesson was that AI-generated output must be verified rather than trusted. Some generated report text described test results and performance figures that I had never measured, so I removed them and kept only results supported by my own screenshots. I also learned to change one thing at a time and confirm each fix with a repeatable request.

If I continued the project, I would add Postman tests for every venue, user and event endpoint, test the ticket-capacity limit with concurrent bookings, and add automated tests so later changes cannot silently break earlier behaviour.

Contribution record

- **Source code:** backend/src/Api.php, backend/src/Middleware.php, backend/.htaccess, frontend/*.html and frontend/assets/js/*.js
- **Database:** database/schema.sql (DDL) and database/seed.sql (DML)
- **Repository:** https://github.com/stomeyamzar/EventEase-Project-SWC3633 — initial upload followed by separate commits for .htaccess, the environment template, the backend .gitignore and log-folder protection

- **Testing:** docs/EventEase.postman_collection.json, with results in Section 4 and Appendices A–B
- **Demonstration:** https://youtu.be/R7eWpEWpOoc

Appendix A: Postman API Testing Evidence

The following screenshots show selected responses captured during local testing of the EventEase API. Status codes and response bodies are visible in each screenshot. Login token screenshots are intentionally excluded to avoid exposing bearer credentials.

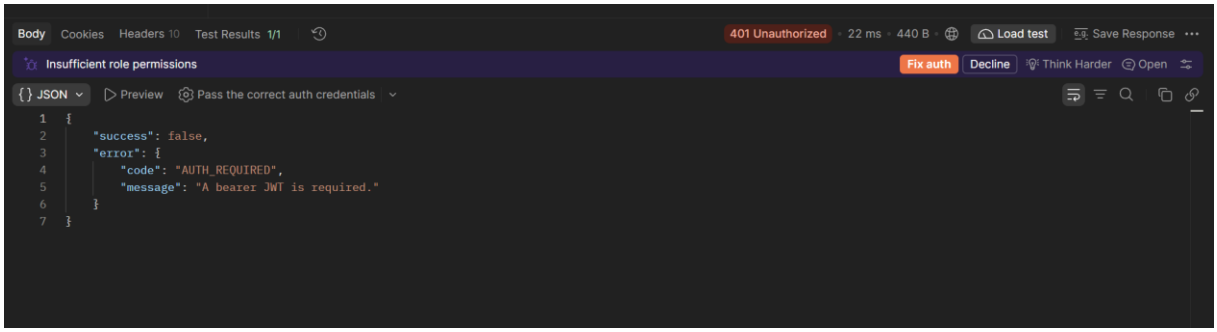
Public events list — 200 OK



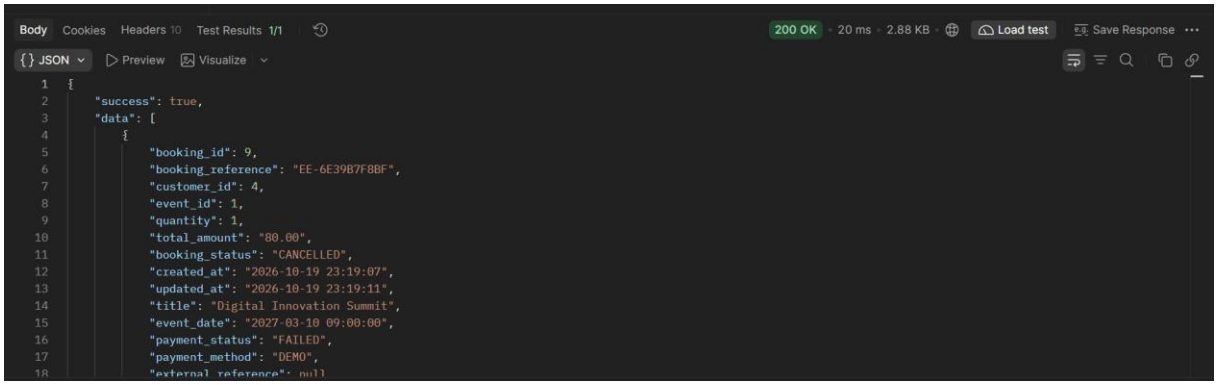
Public search/filter result — 200 OK



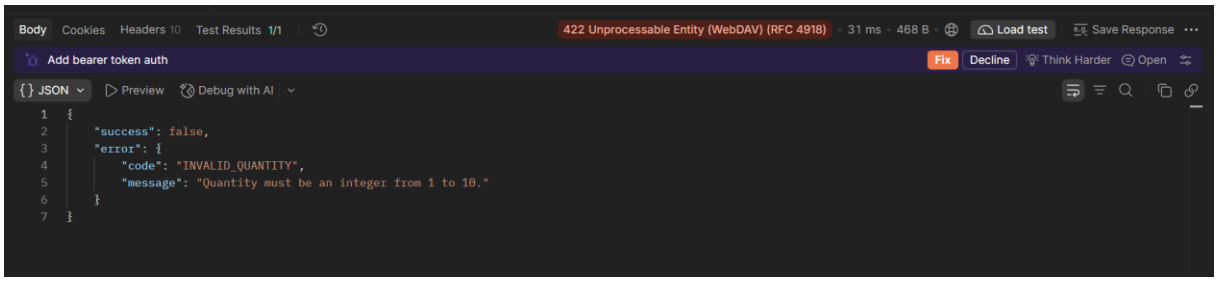
Protected endpoint without JWT — 401 Unauthorized



My bookings response — 200 OK



Invalid booking quantity — 422 Unprocessable Entity



Registration with invalid email — 422 Unprocessable Entity



Admin metrics with API key — 200 OK



Appendix B — Latest Postman Test Evidence (20 October 2026)

This appendix records the latest verification performed in Postman. Screenshots are included as supporting evidence. Tokens are not reproduced in this report.

Test	Observed result	Evidence / meaning
Admin login	200 OK	A fresh administrator JWT was issued successfully.
Admin metrics with valid API key	200 OK	Returned users: 5, events: 6, bookings: 11. The Postman test passed.
Admin metrics with incorrect API key	401 Unauthorized	The API rejected an incorrect X-API-Key.
Bookings with invalid/expired JWT	401 Unauthorized	The API rejected an invalid or expired JWT.
Bookings with fresh admin JWT	200 OK	The bookings endpoint returned booking records; Postman test passed 1/1.

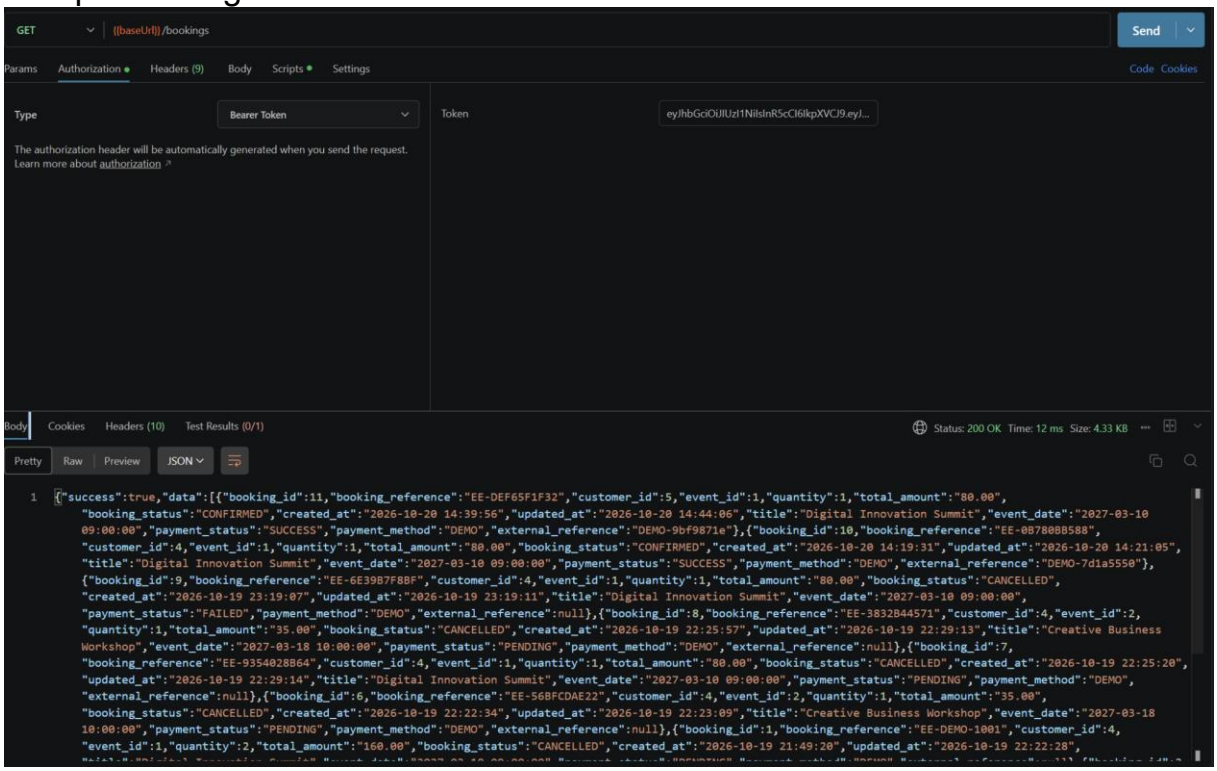
Admin metrics with the valid API key — 200 OK



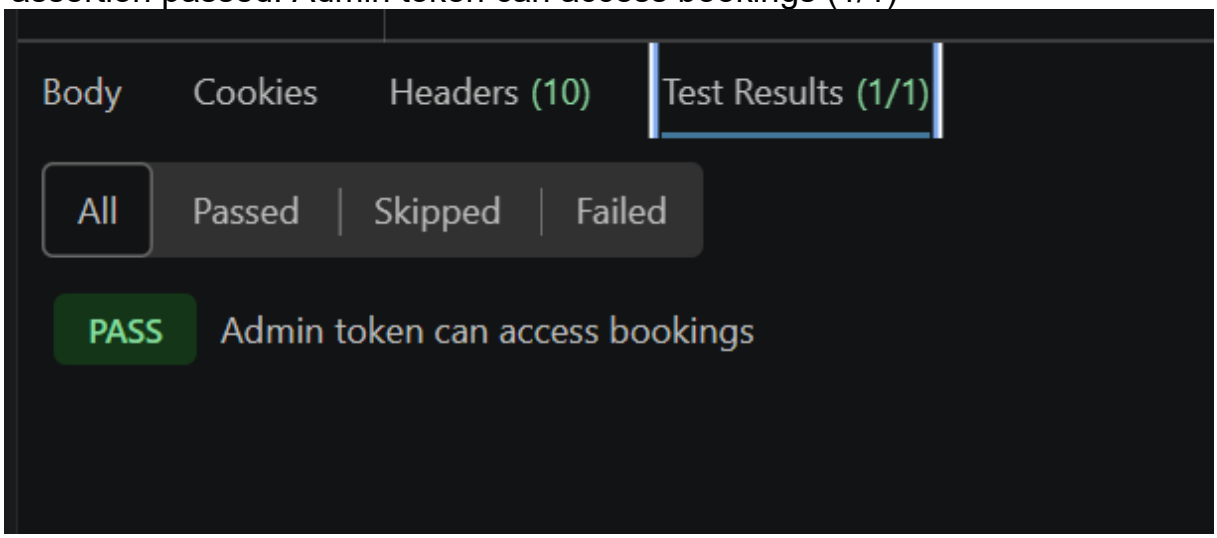
Invalid/expired JWT — 401 Unauthorized



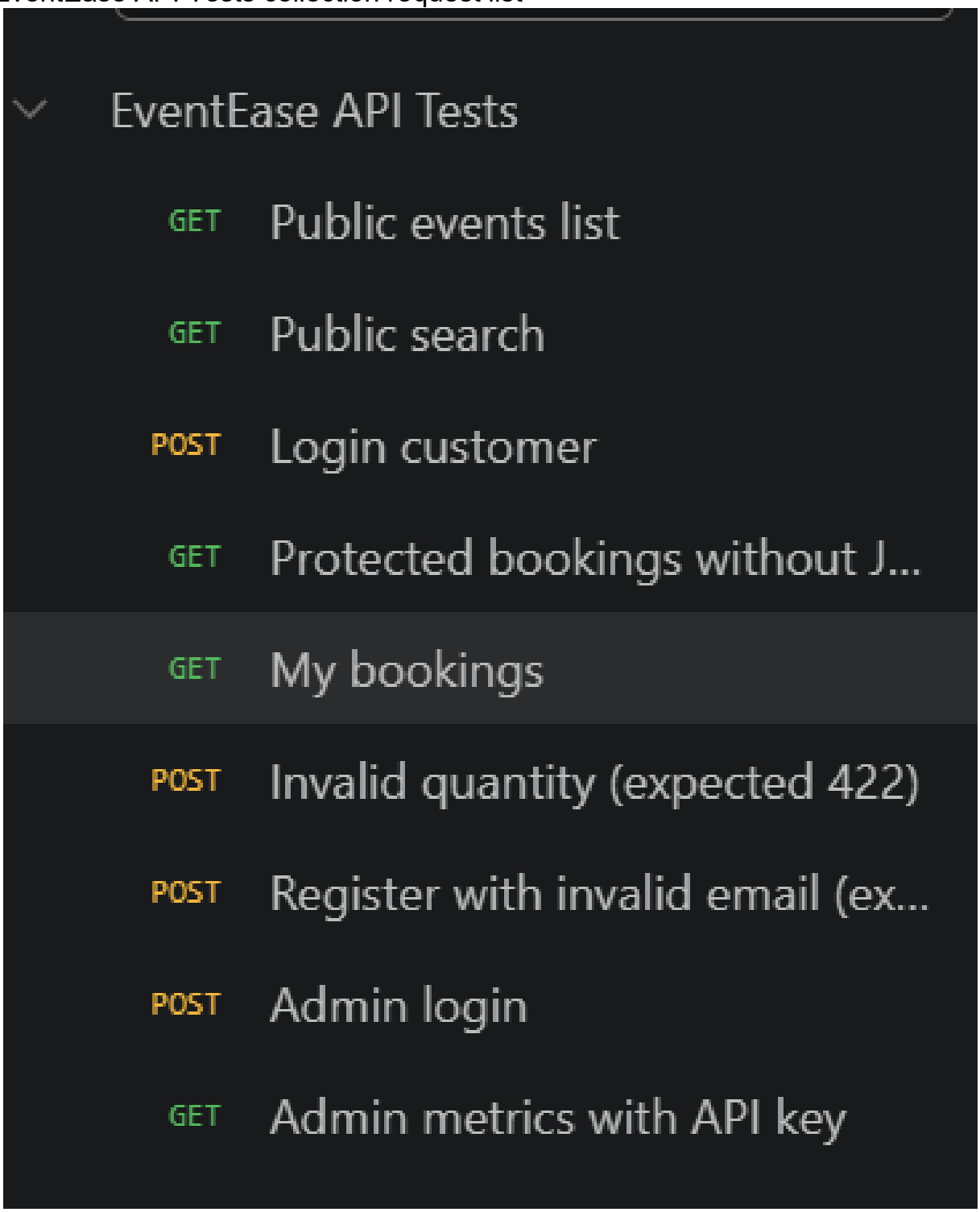
Bookings endpoint using the fresh administrator token — 200 OK



Postman assertion passed: Admin token can access bookings (1/1)



Current EventEase API Tests collection request list



Note: The screenshots above show selected current results. The full collection contains nine requests. Older test evidence remains in Appendix A. Before final submission, replace any remaining personal-detail or reflection placeholders and verify that the SQL export contains the latest sample data.